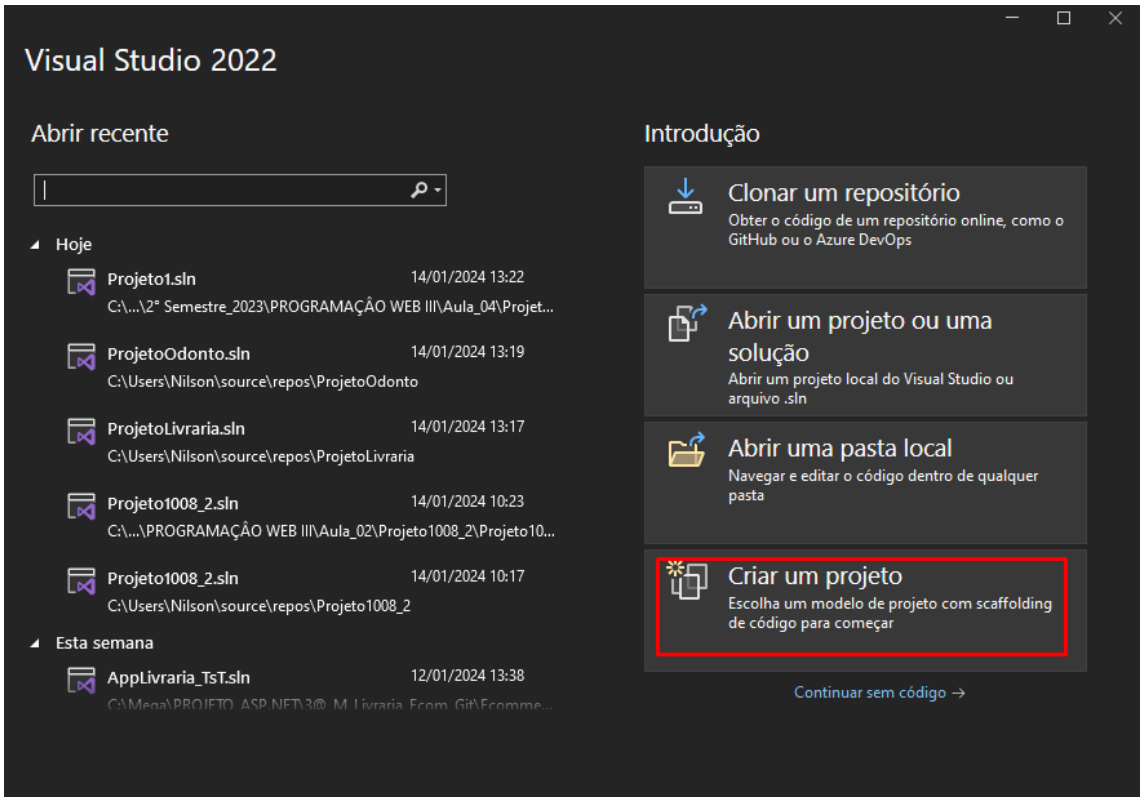
Projeto empréstimo de livros.

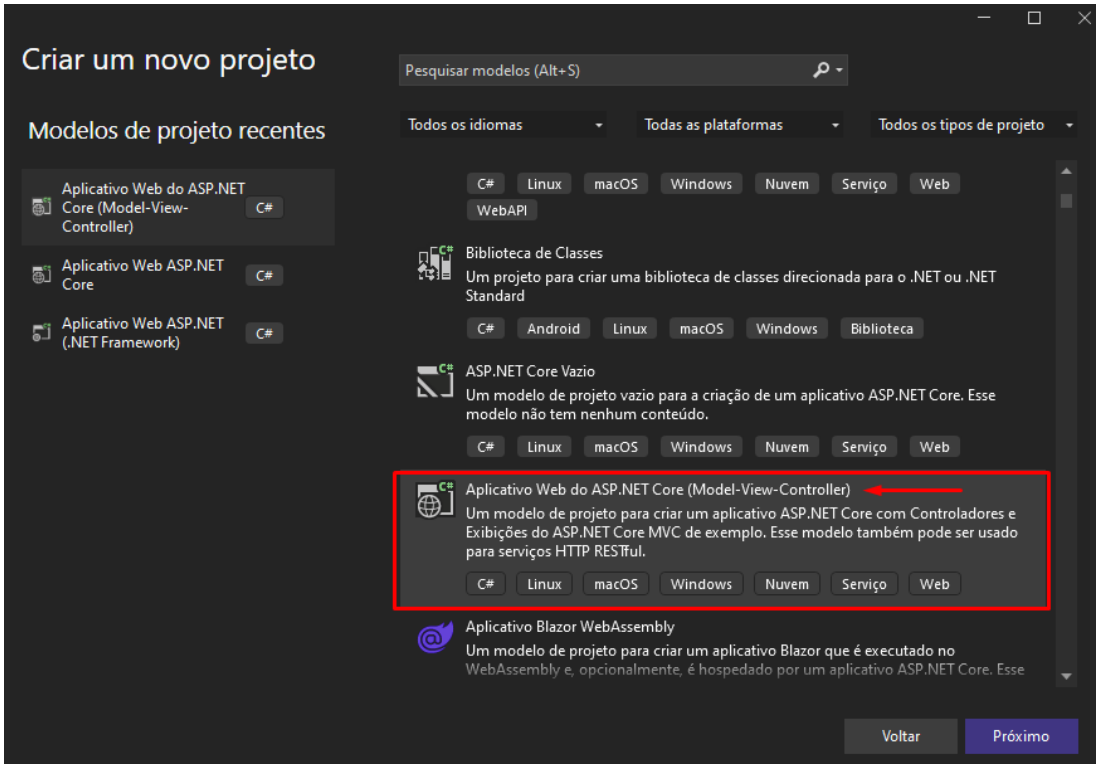
Vamos criar um banco de dados empréstimo

```sql
create database bdEmprestimo;
use bdEmprestimo;

create table tbUsuario(
codUsu int primary key auto_increment,
nomeUsu varchar(50)
);

create table tbLivro(
codLivro int primary key auto_increment,
nomeLivro varchar(50),
imagemLivro varchar(255)
);


create table tbEmprestimo(
codEmp int primary key auto_increment,
dataEmp varchar(20),
dataDev varchar(20),
codUsu int references tbUsuario(codUsu)
);

create table itensEmp(
codItem int primary key auto_increment,
codEmp int references tbEmprestimo(codEmp),
codLivro int references tbLivro(codLivro)
)

select * from itensEmp;

insert into tbUsuario values (default,'Nilson'),(default,'Bruno')
```

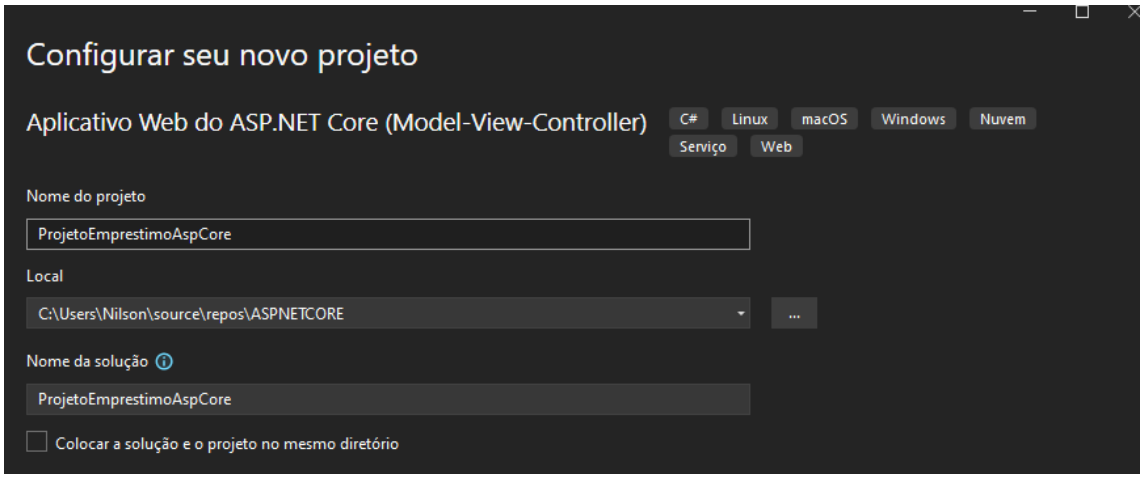
Vamos criar o projeto no Visual Studio 2022



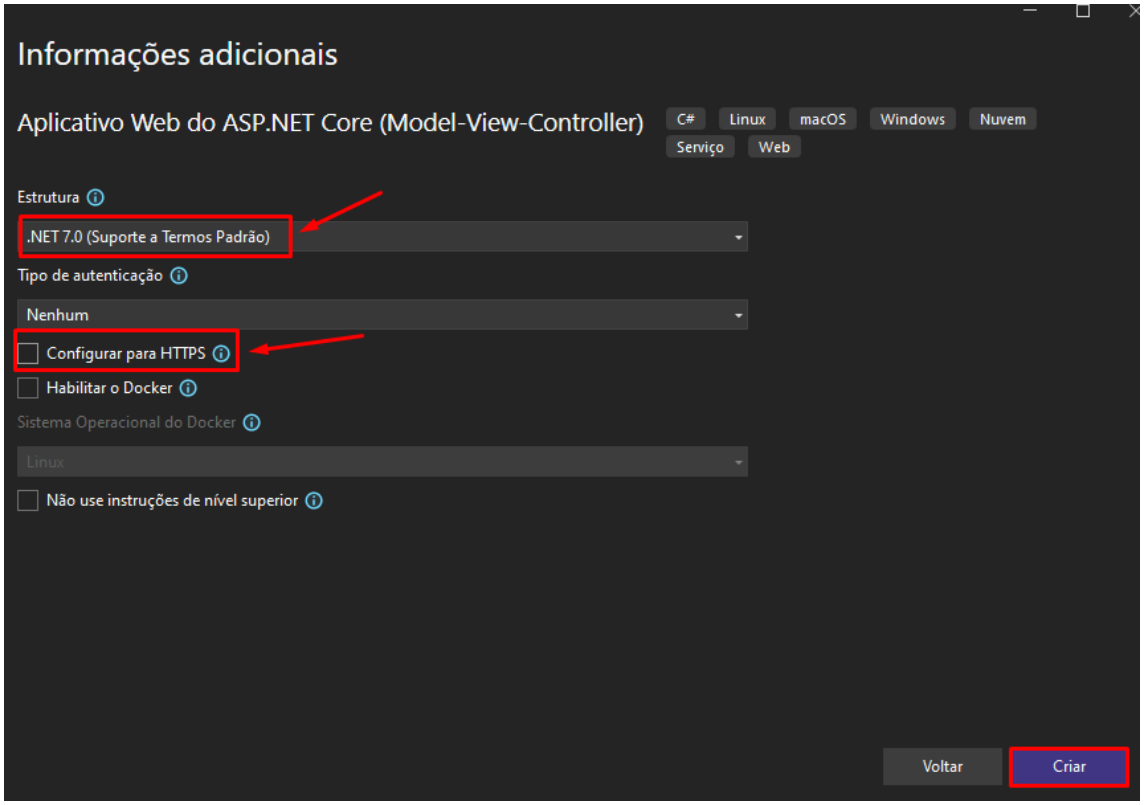
Aplicação Web do ASP.NET Core (Model-View-Controller)



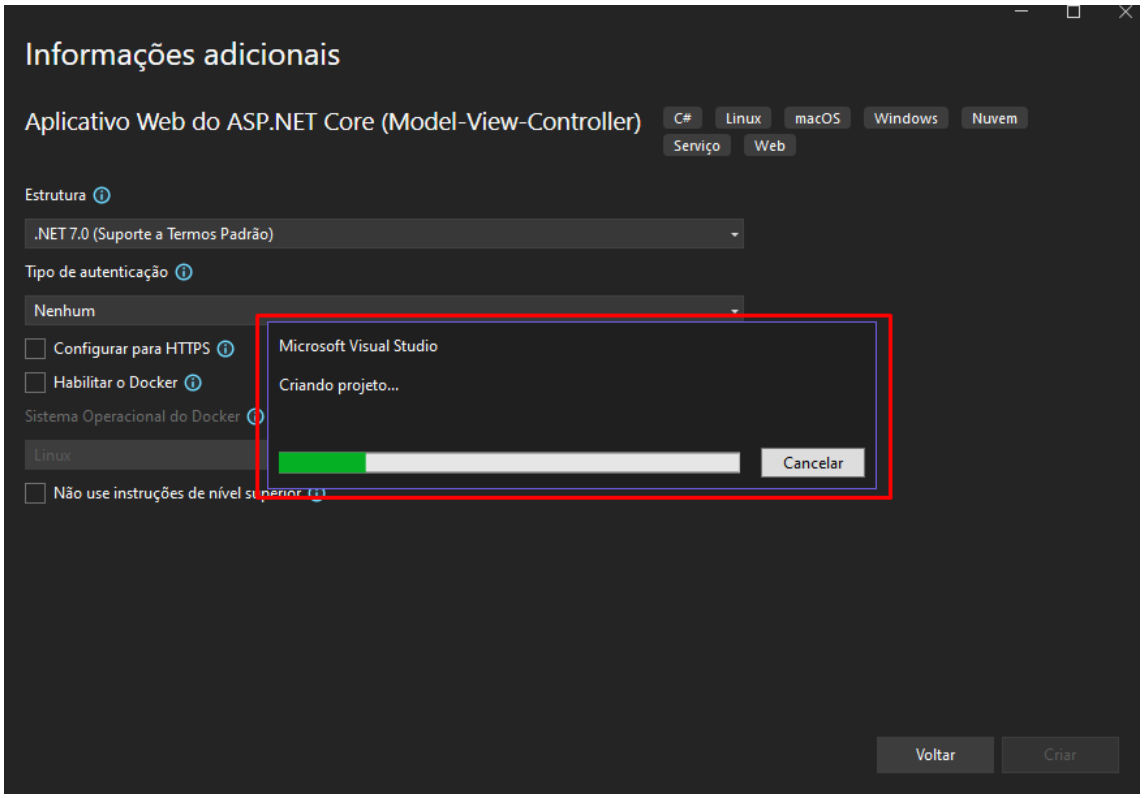
Vamos colocar o nome ProjetoEmprestimoAspCore



Deixar a opção configurar para HTTPS desmarcada depois clica em Criar



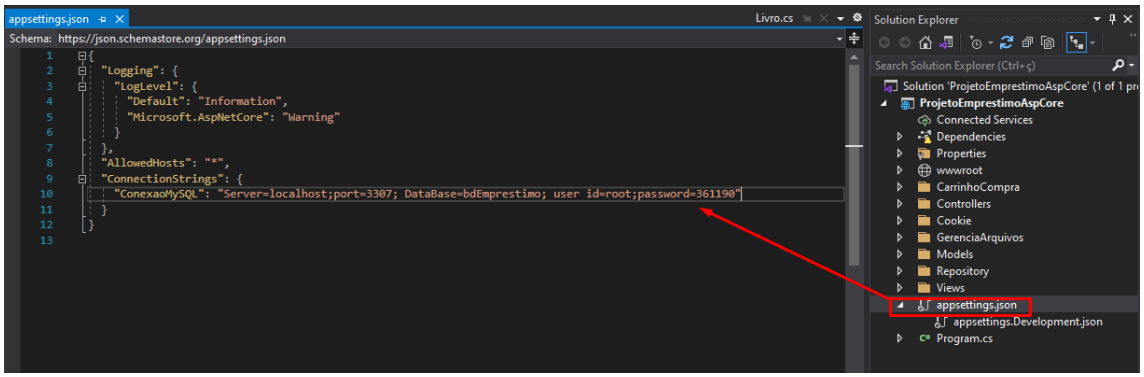
O processo de criação do projeto iniciará, vamos aguardar ....



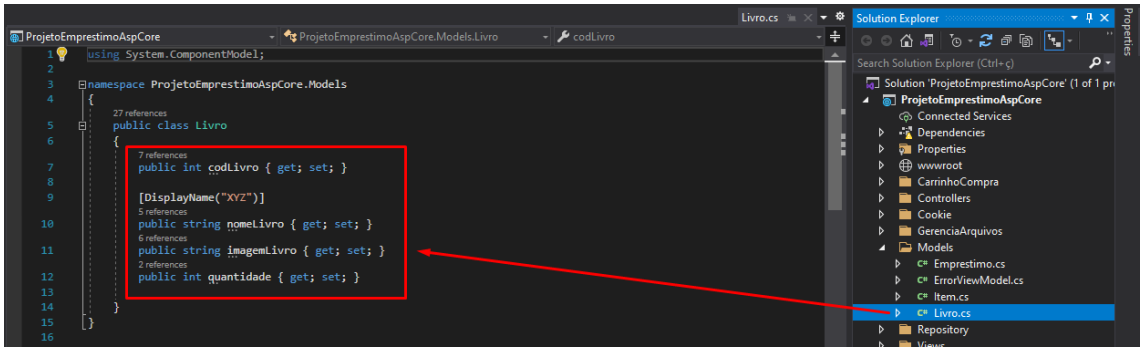
Pronto. Após o projeto ser criado vamos aguardar carregar todas as dependências.

Vamos executar o nosso projeto, sem ter add nenhum arquivo, para testar se está funcionando. * Dica * – Sempre faça este procedimento.

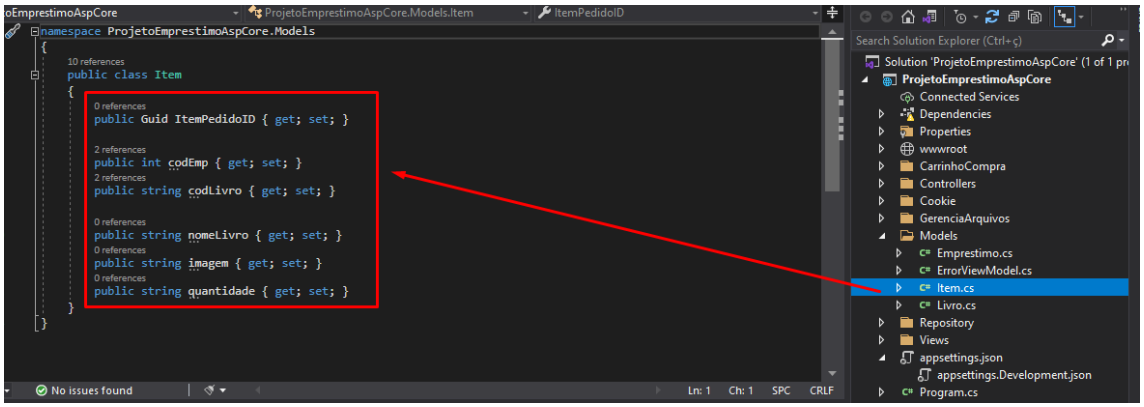
Vamos adicionar a String de conexão com MySql



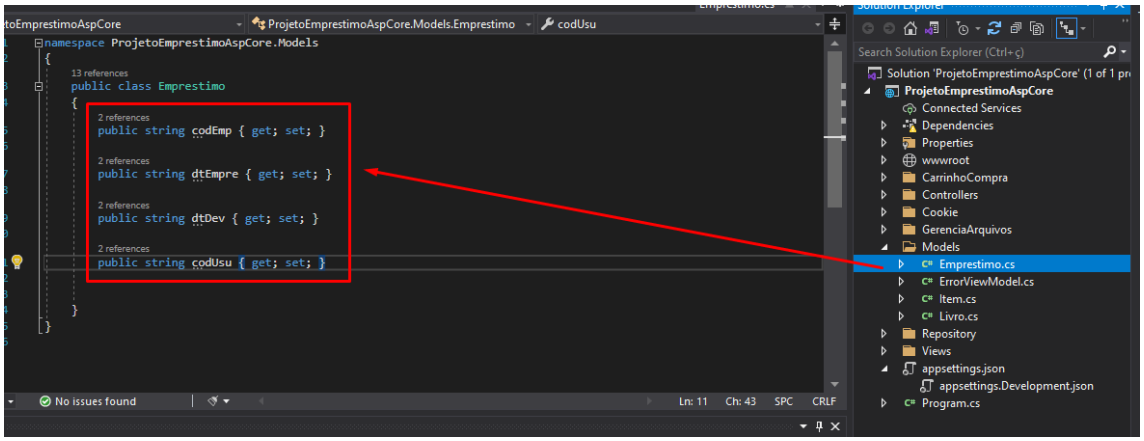
Adicione uma classe na pasta Model com o nome Livro



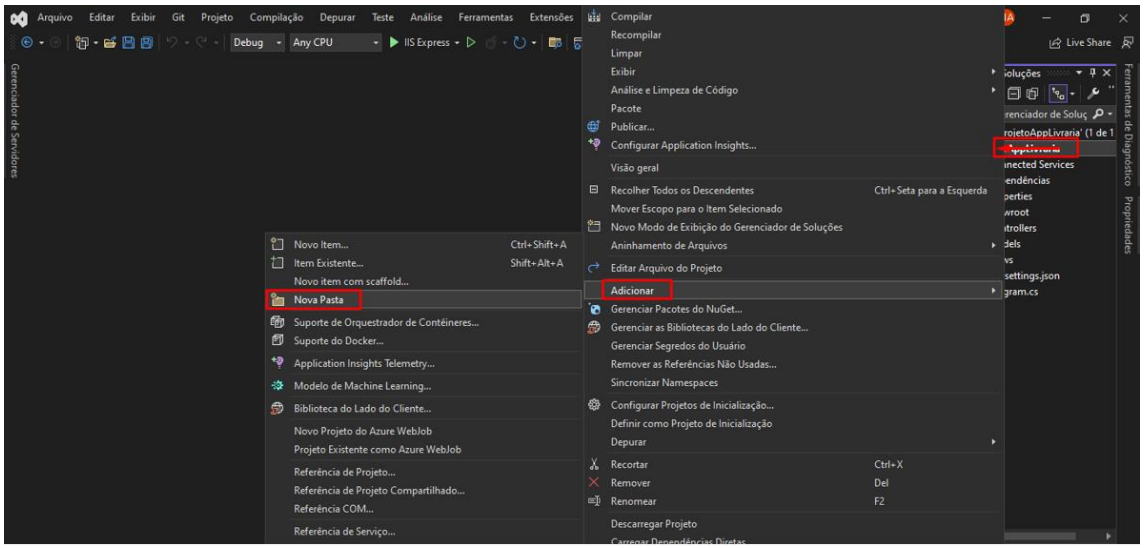
Adicione uma classe na pasta Model com o nome Itens



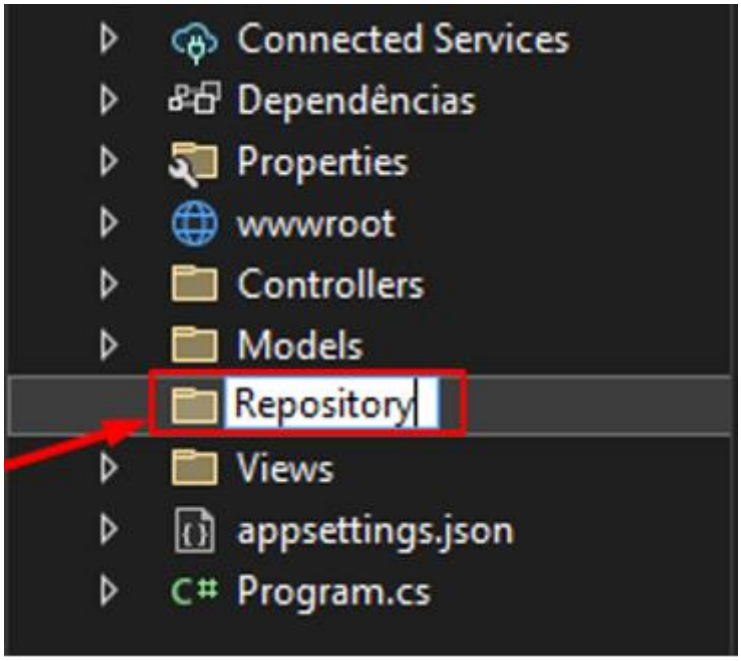
Adicione uma classe na pasta Model com o nome Emprestimo



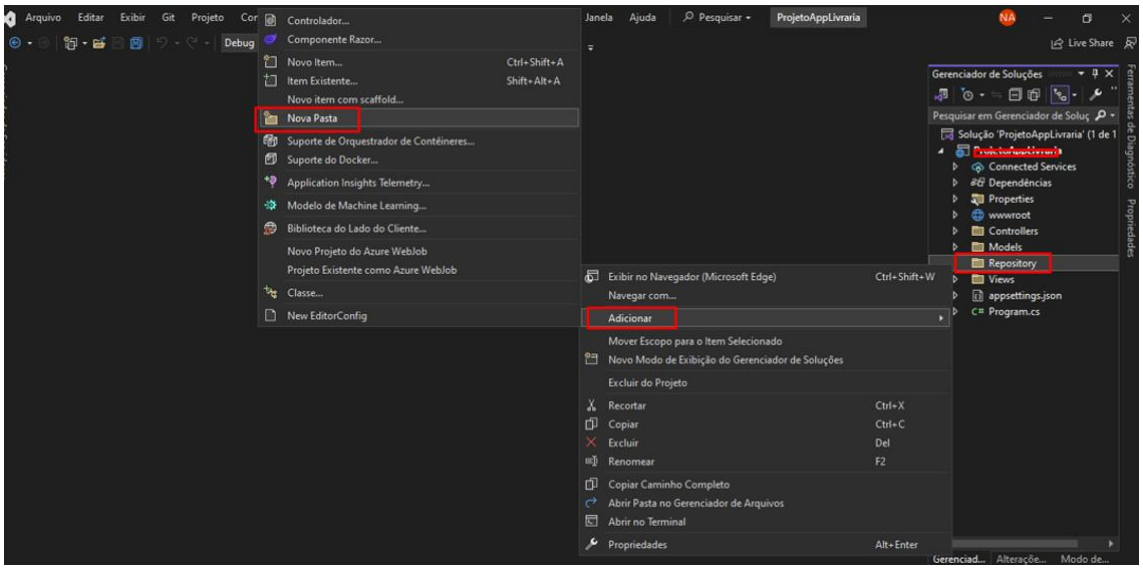
Clica com botão direito sobre o projeto – Adicionar – nova pasta



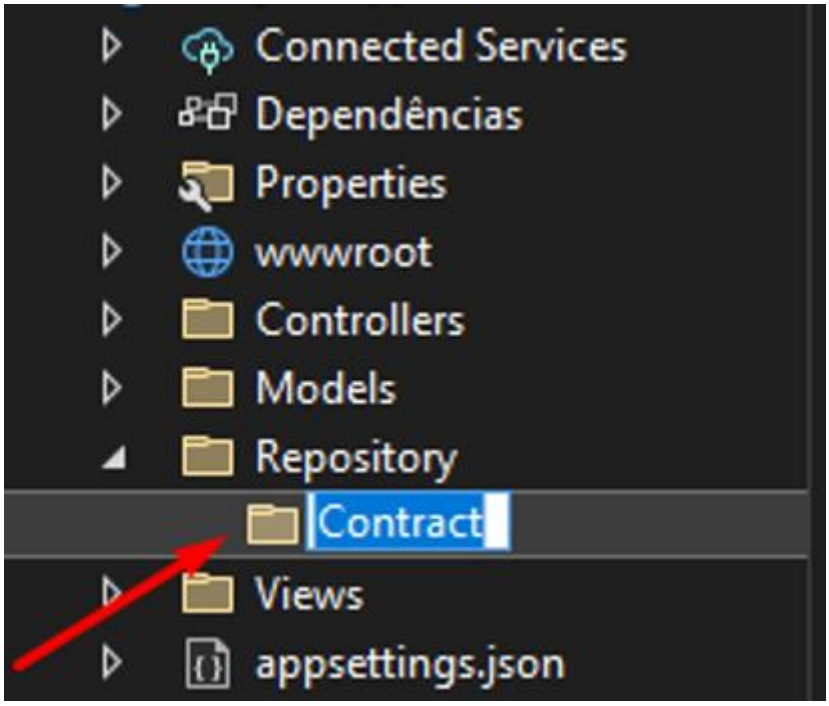
Vamos renomear para repository



Clicando com o botão direito sobre a pasta Repository

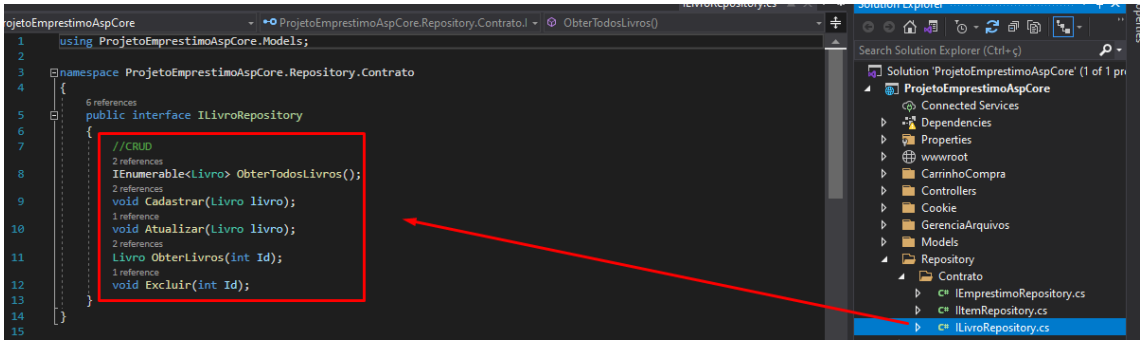


Vamos colocar o nome de Contract



Agora vamos adicionar um Interface chamada ILivroRepository. Que é uma classe abstrata, nesta classe ficará somente a assinatura dos métodos.

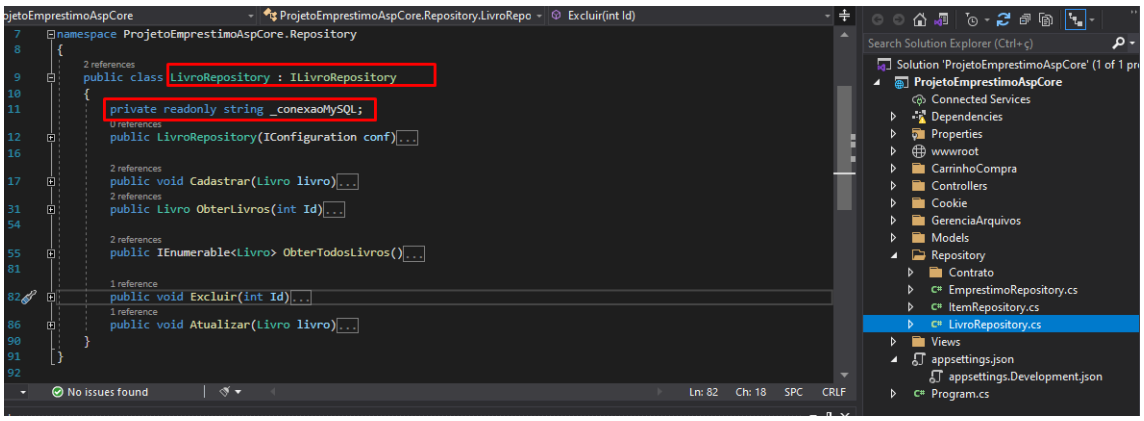
- Clicando com o botão direito sobre a pasta Contract, depois em adicionar – Classe



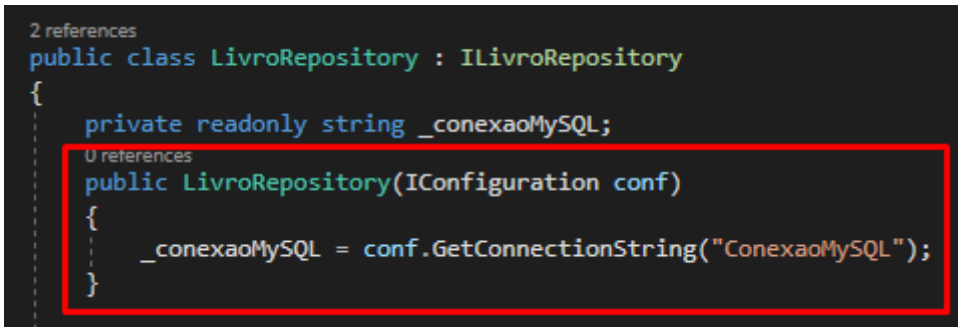
Agora vamos adicionar uma classe concreta (LivroRepository), que implementa os métodos da nossa classe abstrata.

Após criar a classe vamos colocar um Extends para interface

Depois colocar a conexão com o banco de dados, passando uma propriedade privada.



Vamos usar a conexão como parâmetro no construtor da classe.



Vamos implementar os métodos da interface e depois reescrever os métodos

```
2 references
public class LivroRepository : ILivroRepository
{
    private readonly string _conexaoMySQL;
    0 references
    public LivroRepository(IConfiguration conf)...

    2 references
    public void Cadastrar(Livro livro)...
    2 references
    public Livro ObterLivros(int Id)...

    2 references
    public IEnumerable<Livro> ObterTodosLivros()...

    1 reference
    public void Excluir(int Id)...
    1 reference
    public void Atualizar(Livro livro)...
}
```

Vamos criar um método para cadastrar o livro no LivroRepository

```
2 references
public void Cadastrar(Livro livro)
{
    using (var conexao = new MySqlConnection(_conexaoMySQL))
    {
        conexao.Open();

        MySqlCommand cmd = new MySqlCommand("insert into tbLivro values(default, @NomeLivro, @ImagemLivro)", conexao);

        cmd.Parameters.Add("@NomeLivro", MySqlDbType.VarChar).Value = livro.nomeLivro;
        cmd.Parameters.Add("@ImagemLivro", MySqlDbType.VarChar).Value = livro.imagemLivro;
        cmd.ExecuteNonQuery();
        conexao.Close();
    }
}
```

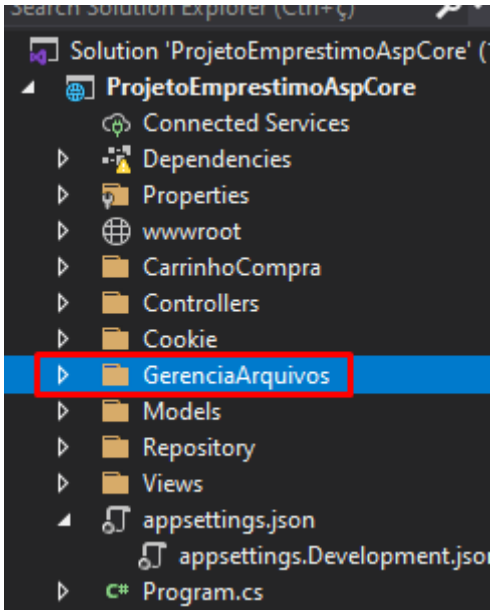
Vamos criar um método para obter os livros por ID na classe LivroRepository

```
2 references
public Livro ObterLivros(int Id)
{
    using (var conexao = new MySqlConnection(_conexaoMySQL))
    {
        conexao.Open();

        MySqlCommand cmd = new MySqlCommand("select * from tbLivro where codLivro=@cod", conexao);
        cmd.Parameters.Add("@cod", MySqlDbType.VarChar).Value = Id;

        MySqlDataAdapter da = new MySqlDataAdapter(cmd);
        MySqlDataReader dr;

        Livro livro = new Livro();
        dr = cmd.ExecuteReader(CommandBehavior.CloseConnection);
        while (dr.Read())
        {
            livro.codLivro = Convert.ToInt32(dr["codLivro"]);
            livro.nomeLivro = (String)(dr["nomeLivro"]);
            livro.imagemLivro = (String)(dr["imagemLivro"]);
        }
        return livro;
    }
}
```

Vamos criar um método para obter todos os livros na classe LivroRepository

```
public IEnumerable<Livro> ObterTodosLivros()
{
    List<Livro> Livrolist = new List<Livro>();
    using (var conexao = new MySqlConnection(_conexaoMySQL))
    {
        conexao.Open();
        MySqlCommand cmd = new MySqlCommand("select * from tbLivro", conexao);
        MySqlDataAdapter sd = new MySqlDataAdapter(cmd);
        DataTable dt = new DataTable();

        sd.Fill(dt);
        conexao.Close();

        foreach (DataRow dr in dt.Rows)
        {
            Livrolist.Add(
                new Livro
                {
                    codLivro = Convert.ToInt32(dr["codLivro"]),
                    nomeLivro = (String)(dr["nomeLivro"]),
                    imagemLivro = (String)(dr["imagemLivro"]),
                });
        }
        return Livrolist;
    }
}
```

Vamos criar uma pasta na raiz do projeto com o nome GerenciaArquivo



Dentro vamos colocar um classe GerenciadorArquivo

Dentro desta classe vamos criar um método para criar o caminho da nossa pasta para salvar as imagens

```
namespace ProjetoEmprestimoAspCore.GerenciaArquivos
{
    public class GerenciadorArquivo
    {
        public static string CadastrarImagemProduto(IFormFile file)
        {
            var NomeArquivo = Path.GetFileName(file.FileName);
            var Caminho = Path.Combine(Directory.GetCurrentDirectory(), "wwwroot/Imagens", NomeArquivo);

            using (var stream = new FileStream(Caminho, FileMode.Create))
            {
                file.CopyTo(stream);
            }

            return Path.Combine("/Imagens", NomeArquivo).Replace("\\", "/");
        }
    }
}
```

Vamos adicionar um LivroController

No Controller vamos primeiramente injetar o livro ILivroRepository e usar no construtor

Depois vamos adicionar um método que cadastra o livro

```
1 reference
public class LivrosController : Controller
{

    private ILivroRepository _livroRepository;

    0 references
    public LivrosController(ILivroRepository livroRepository)
    {
        _livroRepository = livroRepository;
    }
    0 references
    public IActionResult Index()
    {
        return View();
    }
    [HttpPost]
    0 references
    public IActionResult Index(Livro livro, IFormFile file)
    {
        var Caminho = GerenciadorArquivo.CadastrarImagemProduto(file);

        livro.imagemLivro = Caminho;

        _livroRepository.Cadastrar(livro);

        ViewBag.msg = "Cadastro realizado";
        return View();
    }
}
```

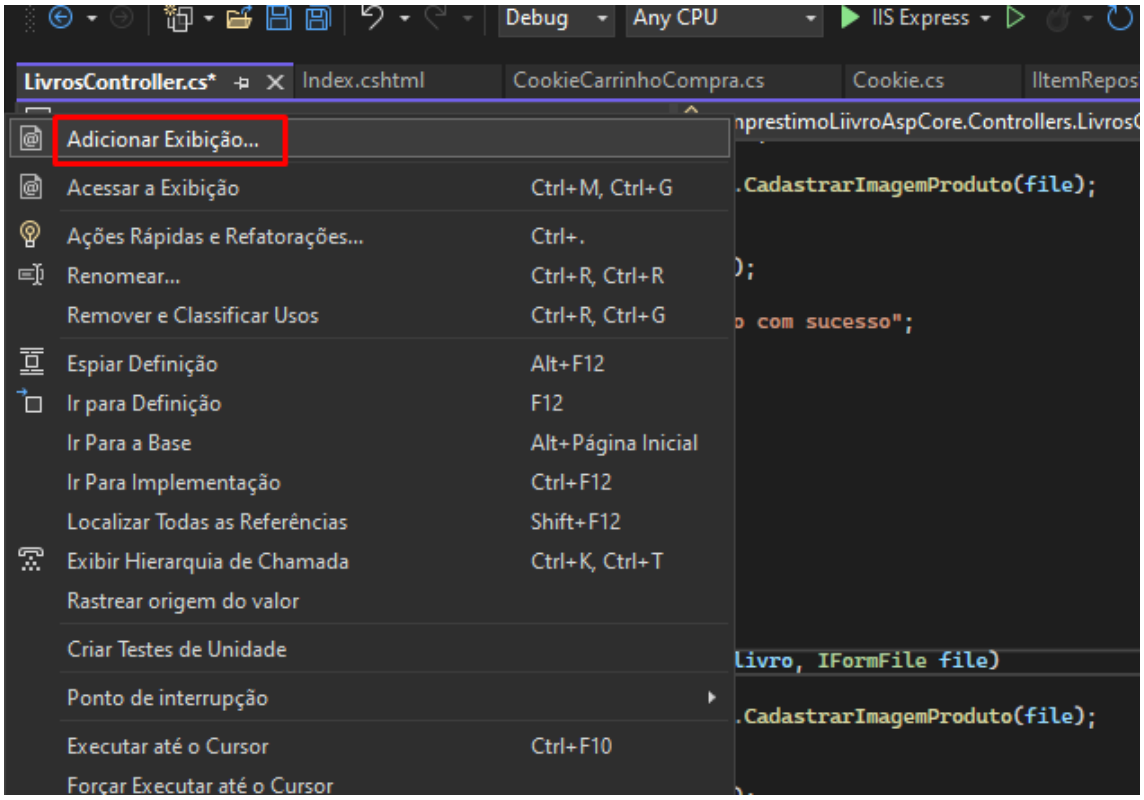
Vamos criar a view que cadastra o livro usando a Index

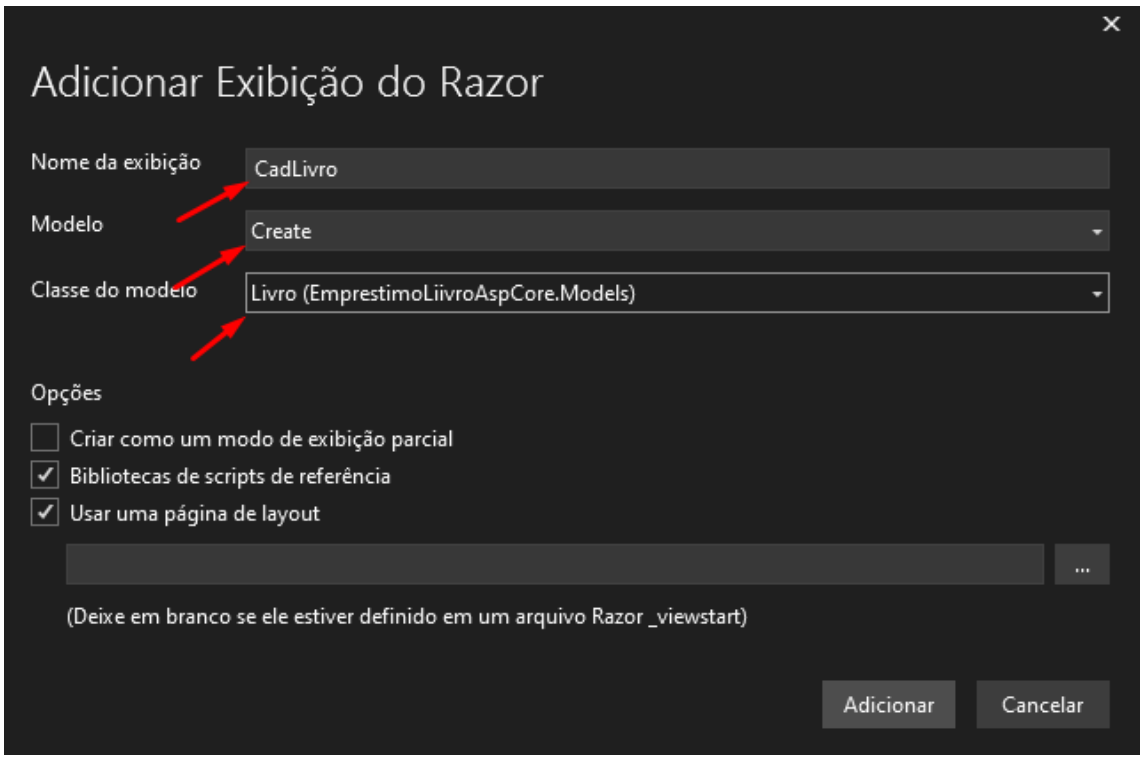
```
@model ProjetoEmprestimoAspCore.Models.Livro
@{
    ViewBag.Title = "Cadastrar Novo Livro";
}
<h2 style="text-align:center;">Novo Livro</h2>

<div class="center">
    @using (Html.BeginForm("Index", "Livros", FormMethod.Post, new { enctype = "multipart/form-data" }))
    {
        @Html.AntiForgeryToken()

        <div class="form-horizontal">
            <hr />
            @Html.ValidationSummary(true, "", new { @class = "text-danger" })
            <div class="form-group space-around-flex">
                <div class="">
                    @Html.LabelFor(model => model.nomeLivro, htmlAttributes: new { @class = "control-label col-md-2" })
                </div>
                <div class="col-md-10">
                    @Html.EditorFor(model => model.nomeLivro, new { htmlAttributes = new { @class = "form-control" } })
                    @Html.ValidationMessageFor(model => model.nomeLivro, "", new { @class = "text-danger" })
                </div>
            </div>
            <div class="form-group center">
                <div>
                    <label class="control-label col-md-2" for="file">Capa Livro</label>
                </div>
                <div class="col-md-10">
                    <input type="file" name="file" id="file" /> <br />
                </div>
            </div>
```

```
            <div class="form-group center">
                <div class="col-md-10">
                    @ViewBag.msg <br />
                </div>
            </div>

            <div class="form-group">
                <div class="col-md-offset-2 col-md-10">
                    <input type="submit" value="Salvar Produto" class="btn btn-success" />
                </div>
            </div>
        </div>
    }
```

Vamos adicionar a interface como serviço na classe Program.cs

```
Program.cs*   HomeController.cs   LivrosController.cs   LivroRepository.cs
ProjetoEmprestimoAspCore

4   using ProjetoEmprestimoAspCore.Repository.Contrato;
5
6   var builder = WebApplication.CreateBuilder(args);
7
8   // Add services to the container.
9   builder.Services.AddControllersWithViews();
10
11  builder.Services.AddHttpContextAccessor();
12
13  // Adicionar um serviço para Injetar os metodos no Controller
14  builder.Services.AddScoped<ILivroRepository, LivroRepository>();
```


Debug
Any CPU
IIS Express
LivrosController.cs*
Index.cshtml
CookieCarrinhoCompra.cs
Cookie.cs
IItemRepos
Adicionar Exibição...
Acessar a Exibição Ctrl+M, Ctrl+G
Ações Rápidas e Refatorações... Ctrl+.
Renomear... Ctrl+R, Ctrl+R
Remover e Classificar Usos Ctrl+R, Ctrl+G
Espiar Definição Alt+F12
Ir para Definição F12
Ir Para a Base Alt+Página Inicial
Ir Para Implementação Ctrl+F12
Localizar Todas as Referências Shift+F12
Exibir Hierarquia de Chamada Ctrl+K, Ctrl+T
Rastrear origem do valor
Criar Testes de Unidade
Ponto de interrupção
Executar até o Cursor Ctrl+F10
Forçar Executar até o Cursor
.CadastrarImagemProduto(file);
livro, IFormFile file)


Adicionar Exibição do Razor
Nome da exibição
CadLivro
Modelo
Create
Classe do modelo
Livro (EmprestimoLiivroAspCore.Models)
Opções
Criar como um modo de exibição parcial
Bibliotecas de scripts de referência
Usar uma página de layout
(Deixe em branco se ele estiver definido em um arquivo Razor _viewstart)
Adicionar
Cancelar

```
ml    CadLivro.cshtml*    LivrosController.cs    Index.cshtml    CookieCarrinhoCompra.cs    Cookie.cs    IItemRepositor
timoLiivroAspCore
@model EmprestimoLiivroAspCore.Models.Livro

@{
    ViewData["Title"] = "CadLivro";
}

<h1>CadLivro</h1>

<h4>Livro</h4>
<hr />
<div class="row">
    <div class="col-md-4">
        <form asp-action="CadLivro" asp-controller="Livros" method="post" enctype="multipart/form-data">
            <div asp-validation-summary="ModelOnly" class="text-danger"></div>
            <div class="form-group">
                <input asp-for="codLivro" type="hidden" class="form-control" />
                <span asp-validation-for="codLivro" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="nomeLivro" class="control-label"></label>
                <input asp-for="nomeLivro" class="form-control" />
                <span asp-validation-for="nomeLivro" class="text-danger"></span>
            </div>
```

```
            <br />
            <div class="form-group">

                <input type="file" name="file" id="file" /> <br />

                <span asp-validation-for="imagemLivro" class="text-danger"></span>
            </div>
            @*VAMOS ADD ESTA MENSAGEM*@
             @ViewBag.msg
             <br />
            <div class="form-group">
                <input type="submit" value="Cadastrar" class="btn btn-success" />
            </div>
        </form>
    </div>
</div>

<div>
    <a asp-action="Index">Back to List</a>
</div>

@section Scripts {
    @{await Html.RenderPartialAsync("_ValidationScriptsPartial");}
}
```

Vamos criar a interface de IItemRepository

Clicando com o botão direito sobre a pasta Contract, depois em adicionar – Classe

```
using ProjetoEmprestimoAspCore.Models;

namespace ProjetoEmprestimoAspCore.Repository.Contrato
{
    4 references
    public interface IItemRepository
    {
        //CRUD
        1 reference
        IEnumerable<Item> ObterTodosItens();
        2 references
        void Cadastrar(Item item);
        1 reference
        void Atualizar(Item item);
        1 reference
        Item ObterItens(int Id);
        1 reference
        void Excluir(int Id);
    }
}
```

Vamos criar uma classe ItemRepository

Vamos colocar um Extends para a Interface e implementar os métodos

```
2 references
public class ItemRepository : IItemRepository
{
    private readonly string _conexaoMySQL;
    0 references
    public ItemRepository(IConfiguration conf)
    {
        _conexaoMySQL = conf.GetConnectionString("ConexaoMySQL");
    }
    2 references
    public void Cadastrar(Item item)...

    1 reference
    public void Atualizar(Item item)...

    1 reference
    public void Excluir(int Id)...

    1 reference
    public Item ObterItens(int Id)...

    1 reference
    public IEnumerable<Item> ObterTodosItens()...
}
```

Vamos primeiramente criar a conexão colocando a propriedade privada,

Depois injetamos no construtor da classe

Depois vamos reescrever o método cadastrar

```
namespace ProjetoEmprestimoAspCore.Repository
{
    2 references
    public class ItemRepository : IItemRepository
    {
        private readonly string _conexaoMySQL;

        public ItemRepository(IConfiguration conf)
        {
            _conexaoMySQL = conf.GetConnectionString("ConexaoMySQL");
        }
        2 references
        public void Cadastrar(Item item)
        {
            using (var conexao = new MySqlConnection(_conexaoMySQL))
            {
                conexao.Open();

                MySqlCommand cmd = new MySqlCommand("insert into itensEmp values(default, @codEmp, @codLivro)", conexao);

                cmd.Parameters.Add("@codEmp", MySqlDbType.VarChar).Value = item.codEmp;
                cmd.Parameters.Add("@codLivro", MySqlDbType.VarChar).Value = item.codLivro;
                cmd.ExecuteNonQuery();
                conexao.Close();
            }
        }
        1 reference
        public void Atualizar(Item item)...
```

Vamos adicionar a interface como serviço na classe Program.cs

```
using ProjetoEmprestimoAspCore.Repository.Contrato;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllersWithViews();

builder.Services.AddHttpContextAccessor();

// Adicionar um serviço para Injetar os metodos no Controller
builder.Services.AddScoped<ILivroRepository, LivroRepository>();
builder.Services.AddScoped<IItemRepository, ItemRepository>();
```

Vamos criar a interface de IEmprestimoRepository

Clicando com o botão direito sobre a pasta Contract, depois em adicionar – Classe

```
using ProjetoEmprestimoAspCore.Models;

namespace ProjetoEmprestimoAspCore.Repository.Contrato
{
    4 references
    public interface IEmprestimoRepository
    {
        //CRUD
        1 reference
        IEnumerable<Emprestimo> ObterTodosEmprestimos();
        2 references
        void Cadastrar(Emprestimo emprestimo);
        1 reference
        void Atualizar(Emprestimo emprestimo);
        1 reference
        Emprestimo ObterEmprestimos(int Id);
        2 references
        void buscaIdEmp(Emprestimo emprestimo);
        1 reference
        void Excluir(int Id);
    }
}
```

Vamos criar a classe concreta e implementar os métodos da interface

```
using System.Data;

namespace ProjetoEmprestimoAspCore.Repository
{
    2 references
    public class EmprestimoRepository : IEmprestimoRepository
    {
        private readonly string _conexaoMySQL;
        0 references
        public EmprestimoRepository(IConfiguration conf)...
        1 reference
        public void Atualizar(Emprestimo emprestimo)...

        2 references
        public void Cadastrar(Emprestimo emprestimo)...
        1 reference
        public void Excluir(int Id)...

        2 references
        public void buscaIdEmp(Emprestimo emprestimo)...

        1 reference
        public IEnumerable<Emprestimo> ObterTodosEmprestimos()...

        1 reference
        public Emprestimo ObterEmprestimos(int Id)...
    }
}
```

Vamos colocar a propriedade privada da conexão e depois injetar no construtor

```
using System.Data;

namespace ProjetoEmprestimoAspCore.Repository
{
    2 references
    public class EmprestimoRepository : IEmprestimoRepository
    {
        private readonly string _conexaoMySQL;
        0 references
        public EmprestimoRepository(IConfiguration conf)
        {
            _conexaoMySQL = conf.GetConnectionString("ConexaoMySQL");
        }
        1 reference
        public void Atualizar(Emprestimo emprestimo)...
```

Vamos reescrever o método para cadastrar o empréstimo.

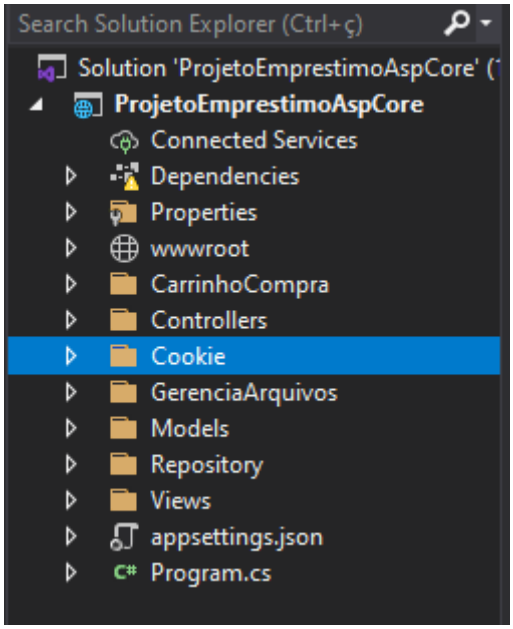
```
namespace ProjetoEmprestimoAspCore.Repository

    2 references
    public class EmprestimoRepository : IEmprestimoRepository
    {
        private readonly string _conexaoMySQL;
        0 references
        public EmprestimoRepository(IConfiguration conf)...

        2 references
        public void Cadastrar(Emprestimo emprestimo)
        {
            using (var conexao = new MySqlConnection(_conexaoMySQL))
            {
                conexao.Open();

                MySqlCommand cmd = new MySqlCommand("insert into tbEmprestimo values(default, @dtEmpre, @dtDev , @codUsu)", conexao);

                cmd.Parameters.Add("@dtEmpre", MySqlDbType.VarChar).Value = emprestimo.dtEmpre;
                cmd.Parameters.Add("@dtDev", MySqlDbType.VarChar).Value = emprestimo.dtDev;
                cmd.Parameters.Add("@codUsu", MySqlDbType.VarChar).Value = emprestimo.codUsu;
                cmd.ExecuteNonQuery();
                conexao.Close();
            }
        }
```

Vamos reescrever o método para buscar o empréstimo por ID.

Esse método pega o ultimo ID que foi inserido no banco

```
epository.cs*  HomeController.cs   LivrosController.cs   LivroRepository.cs
prestimoAspCore   ProjetoEmprestimoAspCore.Repository.EmprestimoRepos   buscaIdEmp(Emprestimo emprestimo)
public class EmprestimoRepository : IEmprestimoRepository
{
    private readonly string _conexaoMySQL;
    0 references
    public EmprestimoRepository(IConfiguration conf)...

    2 references
    public void Cadastrar(Emprestimo emprestimo)...

    2 references
    public void buscaIdEmp(Emprestimo emprestimo)
    {
        using (var conexao = new MySqlConnection(_conexaoMySQL))
        {
            conexao.Open();
            MySqlDataReader dr;

            MySqlCommand cmd = new MySqlCommand("SELECT codEmp FROM tbEmprestimo ORDER BY codEmp DESC limit 1", conexao);
            MySqlDataAdapter da = new MySqlDataAdapter(cmd);
            dr = cmd.ExecuteReader();
            while (dr.Read())
            {
                emprestimo.codEmp = dr[0].ToString();
            }
            conexao.Close();
        }
    }
```

Vamos criar uma pasta na raiz do projeto com nome Cookie



Vamos criar uma classe para manipular os cokie

Primeiramente vamos adicionar

```
private IHttpContextAccessor _context;
private IConfiguration _configuration;
```

Depois injetar no método construtor

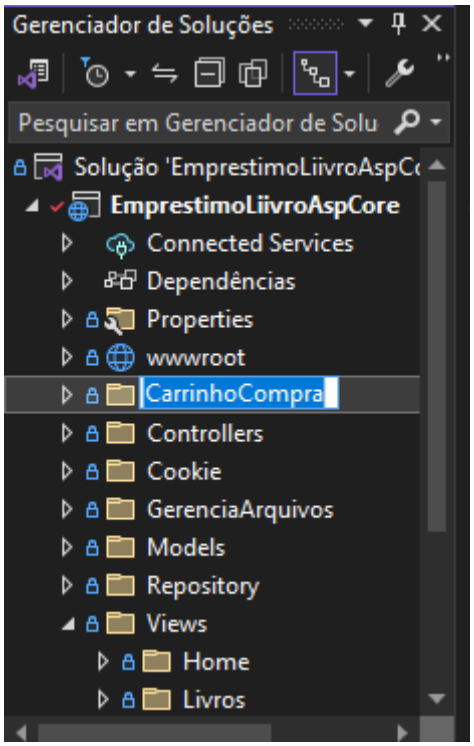
```
0 references
public Cookie(IHttpContextAccessor context, IConfiguration configuration)
{
    _context = context;
    _configuration = configuration;
}
```

```
ojetoEmprestimoAspCore    ProjetoEmprestimoAspCore.Cookie.Cookie    _configuration
1   using Fersoft.NETCore.Cipher;
2
3   namespace ProjetoEmprestimoAspCore.Cookie
4   {
        4 references
5       public class Cookie
6       {
7           private IHttpContextAccessor _context;
8           private IConfiguration _configuration;
9
            0 references
10          public Cookie(IHttpContextAccessor context, IConfiguration configuration)
11          {
12              _context = context;
13              _configuration = configuration;
14          }
15
16          /*
17           * CRUD - Cadastrar/Atualizar/Consultar/Remover - RemoverTodos/Exist
18           */
            2 references
19          public void Cadastrar(string Key, string Valor)...
            0 references
27          public void Atualizar(string Key, string Valor)...
            3 references
35          public void Remover(string Key)...
            1 reference
39          public string Consultar(string Key, bool Cript = true)...
            4 references
44          public bool Existe(string Key)...
            0 references
```

```
// Cadastrar Cookie
2 referências | Nilson Alcides, há 97 dias | 1 autor, 1 alteração
public void Cadastrar(string Key, string Valor)
{
    CookieOptions Options = new CookieOptions();
    Options.Expires = DateTime.Now.AddDays(7);
    Options.IsEssential = true;

    _context.HttpContext.Response.Cookies.Append(Key, Valor, Options);
}
// Deleta Cookie
2 referências | Nilson Alcides, há 97 dias | 1 autor, 1 alteração
public void Remover(string Key)
{
    _context.HttpContext.Response.Cookies.Delete(Key);
}
// Consulta Cookie
1 referência | Nilson Alcides, há 97 dias | 1 autor, 1 alteração
public string Consultar(string Key, bool Cript = true)
{
    var valor = _context.HttpContext.Request.Cookies[Key];
    return valor;
}
```

```
4 referências | Nilson Alcides, há 97 dias | 1 autor, 1 alteração
public bool Existe(string Key)
{
    if (_context.HttpContext.Request.Cookies[Key] == null)
    {
        return false;
    }

    return true; ;
}
0 referências | Nilson Alcides, há 97 dias | 1 autor, 1 alteração
public void Atualizar(string Key, string Valor)
{
    if (Existe(Key))
    {
        Remover(Key);
    }
    Cadastrar(Key, Valor);
}
```

Vamos criar uma pasta CarrinhoCompra



Dentro desta pasta vamos criar uma classe CookieCarrinhoCompra

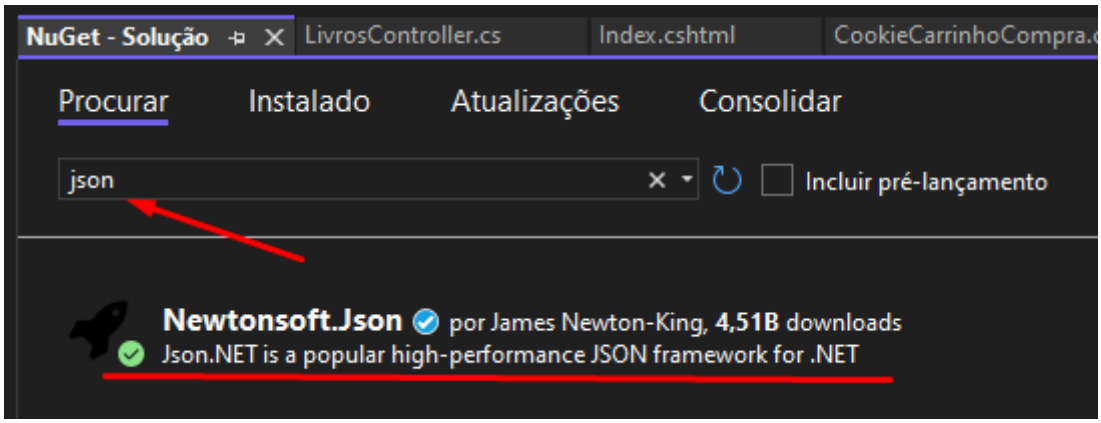
Depois vamos colocar duas propriedades privada

A primeira e a chave e depois injetar a classe cookie

```
//criar uma chave
private string Key = "Carrinho.Compras";
private Cookie.Cookie _cookie;

0 referências | Nilson Alcides, há 97 dias | 1 autor, 1 alteração
public CookieCarrinhoCompra(Cookie.Cookie cookie)
{
    _cookie = cookie;
}
```

Vamos criar os métodos para adicionar os itens no carrinho de compras

Antes vamos usar uma biblioteca Newtonsoft.Json que vai permitir Serializar Deserializar Objeto.



Método para salvar e consultar

```
//Salvar
3 referências | Nilson Alcides, há 97 dias | 1 autor, 1 alteração
public void Salvar(List<Livro> Lista)
{
    string Valor = JsonConvert.SerializeObject(Lista);
    _cookie.Cadastrar(Key, Valor);
}
//Consulta
5 referências | Nilson Alcides, há 97 dias | 1 autor, 1 alteração
public List<Livro> Consultar()
{
    if (_cookie.Existe(Key))
    {
        string valor = _cookie.Consultar(Key);
        return JsonConvert.DeserializeObject<List<Livro>>(valor);
    }
    else
    {
        return new List<Livro>();
    }
}
```

Método para cadastrar o item

```
//Cadastrar
1 referência | Nilson Alcides, há 97 dias | 1 autor, 1 alteração
public void Cadastrar(Livro item)
{
    List<Livro> Lista;
    if (_cookie.Existe(Key))
    {
        Lista = Consultar();
        var ItemLocalizado = Lista.SingleOrDefault(a => a.codLivro == item.codLivro);

        if (ItemLocalizado == null)
        {
            Lista.Add(item);
        }
        else
        {
            ItemLocalizado.quantidade = ItemLocalizado.quantidade + 1;

        }
    }
    else
    {
        Lista = new List<Livro>();
        Lista.Add(item);
    }
    // Criar o metrodo salvar
    Salvar(Lista);
}
```

Método para atualizar os itens e remover

```
//Atualiza
0 referências | Nilson Alcides, há 97 dias | 1 autor, 2 alterações
public void Atualizar(Livro item)
{
    var Lista = Consultar();
    var ItemLocalizado = Lista.SingleOrDefault(a => a.codLivro == item.codLivro);

    if (ItemLocalizado != null)
    {
        ItemLocalizado.quantidade = item.quantidade + 1;
        Salvar(Lista);
    }
}
// remove item
1 referência | Nilson Alcides, há 97 dias | 1 autor, 1 alteração
public void Remover(Livro item)
{
    var Lista = Consultar();
    var ItemLocalizado = Lista.SingleOrDefault(a => a.codLivro == item.codLivro);

    if (ItemLocalizado != null)
    {
        Lista.Remove(ItemLocalizado);
        Salvar(Lista);
    }
}
```

Método para verificar os itens e remover todos

```
// Verifica se existe
0 referências | Nilson Alcides, há 97 dias | 1 autor, 1 alteração
public bool Existe(string Key)
{
    if (_cookie.Existe(Key))
    {
        return false;
    }

    return true;
}
// Remove todos itens do carrinho
1 referência | Nilson Alcides, há 97 dias | 1 autor, 2 alterações
public void RemoverTodos()
{
    _cookie.Remover(Key);
}
```

Agora vamos exibir os itens na Home Index

Acessando o HomeCoontroller vamos criar as propriedades privadas para injetar no método construtor da classe

```
estimoLiivroAspCore                                  EmprestimoLiivroAspCore.Controlle
1   using EmprestimoLiivroAspCore.CarrinhoCompra;
2   using EmprestimoLiivroAspCore.Models;
3   using EmprestimoLiivroAspCore.Repository;
4   using EmprestimoLiivroAspCore.Repository.Contrato;
5   using Microsoft.AspNetCore.Mvc;
6   using System.Diagnostics;
7
8   namespace EmprestimoLiivroAspCore.Controllers
9   {
        1 referência | Nilson Alcides, há 97 dias | 1 autor, 7 alterações
10      public class HomeController : Controller
11      {
12          private ILivroRepository _livroRepository;
13          private CookieCarrinhoCompra _cookieCarrinhoCompra;
14
15          private IEmprestimoRepository _emprestimoRepository;
16          private IItemRepository _itemRepository;
```

Vamos criar o construtor do HomeController

```
namespace EmprestimoLiivroAspCore.Controllers
{
    1 referência | Nilson Alcides, há 97 dias | 1 autor, 7 alterações
    public class HomeController : Controller
    {
        private IItemRepository _itemRepository;
        private IEmprestimoRepository _emprestimoRepository;
        private CookieCarrinhoCompra _cookieCarrinhoCompra;
        private ILivroRepository _livroRepository;

        0 referências | 0 alterações | 0 autores, 0 alterações
        public HomeController(ILivroRepository livroRepository, CookieCarrinhoCompra cookieCarrinhoCompra,
                              IEmprestimoRepository emprestimoRepository, IItemRepository itemRepository)
        {

            _livroRepository = livroRepository;
            _cookieCarrinhoCompra = cookieCarrinhoCompra;
            _emprestimoRepository = emprestimoRepository;
            _itemRepository = itemRepository;
        }
```

Na action index vamos colocar o método que retorna os livros.

```
//Exibe os livros na Index
0 referências | Nilson Alcides, há 97 dias | 1 autor, 2 alterações
public IActionResult Index()
{
    return View(_livroRepository.ObterTodosLivros());
}
```

Vamos criar a Index que exibe os livros

```
prestimoLiivroAspCore
@model IEnumerable<EmprestimoLiivroAspCore.Models.Livro>

@{
    ViewBag.Title = "Biblioteca";
}
<div class="">
    <div class="row center">
        <div class="col-lg-9">
            <div class="row">
                @foreach (var item in Model)
                {
                    <div class="col-lg-3 col-md-6 mb-4" style="margin:2% 4%">
                        <div class="card h-100">

                            @if (@item.imagemLivro != null)
                            {
                                <img class="card-img-right flex-auto d-none d-md-block" data-src=" " alt="Thumbnail [300x300]"
                                style="width: 300px; height: 300px;" src="@item.imagemLivro" data-holder-rendered="true">
                            }
                            else
                            {
                                <img class="card-img-right flex-auto d-none d-md-block" data-src=" " alt="Thumbnail [300x300]"
                                style="width: 300px; height: 300px;" src="~/Images/image-not-found.jpg" data-holder-rendered="true">
                            }

                            <div class="card-body">
                                <h4 class="card-title">
                                    <a href="@item.imagemLivro">@item.nomeLivro</a>
                                </h4>
                                <spam>Código: @item.codLivro</spam>
                            </div>
                            <div class="card-footer space-between-flex" style="width:90%">
                                <div>
                                    @using (Html.BeginForm("AdicionarItem", "Home", FormMethod.Post))
                                    {
                                        <input type="hidden" name="id" value=@item.codLivro>
                                        <input type="hidden" name="quantidade" value="1">
                                        <button type="submit" class="btn btn-default btn-acao-primario">Adicionar</button>
                                    }
                                </div>
                            </div>
                        </div>
                    </div>
                }
```

Vamos criar uma Action AdicionarItem

```
32
33          //Item ID = ID Produto
            0 referências | Nilson Alcides, há 97 dias | 1 autor, 2 alterações
34          public IActionResult AdicionarItem(int id)
35          {
36              Livro produto = _livroRepository.ObterLivros(id);
37
38              if (produto == null)
39              {
40                  return View("NaoExisteItem");
41              }
42              else
43              {
44                  var item = new Livro()
45                  {
46                      codLivro = id,
47                      quantidade = 1,
48                      imagemLivro = produto.imagemLivro,
49                      nomeLivro = produto.nomeLivro
50                  };
51                  _cookieCarrinhoCompra.Cadastrar(item);
52
53                  return RedirectToAction(nameof(Carrinho));
54              }
55
```

```
//Carrnho de compra
2 referências | Nilson Alcides, há 97 dias | 1 autor, 1 alteração
public IActionResult Carrinho()
{
    return View(_cookieCarrinhoCompra.Consultar());
}
//Remover itens do carrinho
0 referências | Nilson Alcides, há 97 dias | 1 autor, 1 alteração
public IActionResult RemoverItem(int id)
{
    _cookieCarrinhoCompra.Remover(new Livro() { codLivro = id });
    return RedirectToAction(nameof(Carrinho));
}
```

```
DateTime data;
0 referências | Nilson Alcides, há 97 dias | 1 autor, 1 alteração
public IActionResult SalvarCarrinho(Emprestimo emprestimo)
{
    List<Livro> carrinho = _cookieCarrinhoCompra.Consultar();

    Emprestimo mdE = new Emprestimo();
    Item mdI = new Item();

    data = DateTime.Now.ToLocalTime();

    mdE.dtEmpre = data.ToString("dd/MM/yyyy");
    mdE.dtDev = data.AddDays(7).ToString();
    mdE.codUsu = "1";
    _emprestimoRepository.Cadastrar(mdE);

    _emprestimoRepository.buscaIdEmp(emprestimo);

    for (int i = 0; i < carrinho.Count; i++)
    {

        mdI.codEmp = Convert.ToInt32(emprestimo.codEmp);
        mdI.codLivro = Convert.ToString(carrinho[i].codLivro);

        _itemRepository.Cadastrar(mdI);
    }

    _cookieCarrinhoCompra.RemoverTodos();
    return RedirectToAction("confEmp");
}
```

Confirmar empréstimo

```
0 referências | Nilson Alcides, há 97 dias | 1 autor, 1 alteraçã
public IActionResult confEmp()
{
    return View();
}
```

Vamos criar as Views

Primeiro o carrinho

```
@model IEnumerable<EmprestimoLiivroAspCore.Models.Livro>

@{
    ViewData["Title"] = "Index";
    decimal Subtotal = 0;
}

<div class="container">
    <section id="order">
        <br />
        <br />
        <h2 class="title-doc">Carrinho de compras</h2>
        <div class="alert alert-danger" style="display: none;" role="alert">
        </div>

        <div id="code_cart">
            <div class="card">
                <table class="table table-hover shopping-cart-wrap">
                    <thead class="text-muted">
                        <tr>
                            <th scope="col">Produto</th>
                            <th scope="col" width="120">Quantidade</th>
                            <th scope="col" width="120"></th>
                            <th scope="col" width="200" class="text-right">Ação</th>
                        </tr>
                    </thead>
```

```
                    </thead>
                    <tbody>
                        @foreach (var item in Model)
                        {
                            <tr>
                                <td>
                                    <figure class="media">
                                        <div class="img-wrap">

                                            <img alt="Thumbnail [300x300]" style="width: 40px; height: 40px;" src="@item.imagemLivro
                                        </div>
                                        <figcaption class="media-body">
                                            <h6 class="title text-truncate">@item.nomeLivro</h6>
                                            <dl class="dlist-inline small">
                                                <dt>Codigo Livro: @item.codLivro </dt>
                                                <dd>@* @item.codLivro *@</dd>
                                            </dl>
                                        </figcaption>
                                    </figure>
                                </td>
```

```
                                <td>
                                    <div class="col-auto">
                                        <div class="input-group mb-2 control-inline">
                                            <input type="hidden" class="inputProdutoId" value="@item.codLivro" />
                                            <input type="hidden" class="inputQuantidadeEstoque" value="@item.quantidade" />
                                            <input type="hidden" class="inputValorUnitario" value="R$ 00,00" />

                                            <input type="text" style="width: 60px; text-align: center" readonly="readonly"
                                            class="form-control inputQuantidadeProdutoCarrinho" value="@item.quantidade">
                                        </div>
                                    </div>
                                </td>

                                <td>

                                </td>
                                <td class="text-right">
                                    <a asp-controller="Home" asp-action="RemoverItem" asp-route-id="@item.codLivro" class="btn btn-outline-danger"> × Remove</a>
                                </td>
                            </tr>
                        }
                    </tbody>
                </table>
            </div>
        </div>
    </section>
    <br />
```

```
72          <br />
73          <br />
74      </div>
75
76      <div class="container">
77          <br />
78          <br />
79          <div class="row">
80              <div class="col-md-12">
81                  Deseja adicionar outro? <a asp-controller="Home" asp-action="Index">Clique aqui</a> para continuar ...
82              </div>
83              <br />
84              <br />
85              <div class="offset-md-4 col-md-4">
86                  <a class="btn btn-outline-primary btn-continuar-comprando" asp-action="SalvarCarrinho" asp-controller="Home">
87                      Finalizar o Empréstimo
88                  </a>
89              </div>
90          </div>
91      </div>
92
```

```
@model IEnumerable<EmprestimoLiivroAspCore.Models.Livro>
@{ ViewData["Title"] = "Index"; decimal Subtotal = 0; }

<div class="container">
    <section id="order">
        <br />
        <br />
        <h2 class="title-doc">Carrinho de compras</h2>
        <div class="alert alert-danger" style="display: none;" role="alert"></div>
        <div id="code_cart">
            <div class="card">
                <table class="table table-hover shopping-cart-wrap">
                    <thead class="text-muted">
                        <tr>
                            <th scope="col">Produto</th>
                            <th scope="col" width="120">Quantidade</th>
                            <th scope="col" width="120"></th>
                            <th scope="col" width="200" class="text-right">Ação</th>
                        </tr>
                    </thead>
                    <tbody>
                        @foreach (var item in Model)
                        {
                            <tr>
                                <td>
                                    <figure class="media">
                                        <div class="img-wrap">

                                            <img alt="Thumbnail [300x300]" style="width: 40px; height: 40px;" src="@item.imagemLivro">
                                        </div>
                                        <figcaption class="media-body">
                                            <h6 class="title text-truncate">@item.nomeLivro</h6>
                                            <dl class="dlist-inline small">
                                                <dt>Codigo Livro: @item.codLivro </dt>
                                                <dd>@* @item.codLivro *@</dd>
                                            </dl>
                                        </figcaption>
                                    </figure>
                                </td>
                                <td>
                                    <div class="col-auto">
                                        <div class="input-group mb-2 control-inline">
                                            <input type="hidden" class="inputProdutoId" value="@item.codLivro" />
                                            <input type="hidden" class="inputQuantidadeEstoque" value="@item.quantidade" />
                                            <input type="hidden" class="inputValorUnitario" value="R$ 00,00" />

                                            <input type="text" style="width: 60px; text-align: center" readonly="readonly" class="form-control
                                            inputQuantidadeProdutoCarrinho" value="@item.quantidade">
                                        </div>
                                    </div>
                                </td>
                                <td> </td>
                                <td class="text-right">
                                    <a asp-controller="Home" asp-action="RemoverItem" asp-route-id="@item.codLivro" class="btn
                                    btn-outline-danger"> × Remove</a>
                                </td>
                            </tr>
                        }
                    </tbody>
                </table>
            </div>
        </div>
    </section>
    <br />
    <br />
</div>

<div class="container">
    <br />
    <br />
    <div class="row">
        <div class="col-md-12">
            Deseja adicionar outro? <a asp-controller="Home" asp-action="Index">Clique aqui</a> para continuar ...
        </div>
        <br />
        <br />
        <div class="offset-md-4 col-md-4">
            <a class="btn btn-outline-primary btn-continuar-comprando" asp-action="SalvarCarrinho" asp-controller="Home">
                Finalizar o Empréstimo
            </a>
        </div>
    </div>
</div>
```

View para confirmar o emprestimo

```
mprestimoLiivroAspCore
@{
    ViewBag.Title = "confEmp";
}

<h2>Empréstimo Finalizado</h2>
```

Vamos adicionar os serviços na classe Program.cs

```
stimoLiivroAspCore
using EmprestimoLiivroAspCore.Repository.Contrato;
using EmprestimoLiivroAspCore.Repository;
using EmprestimoLiivroAspCore.GerenciaArquivos;
using System.Net;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllersWithViews();

builder.Services.AddHttpContextAccessor();

//Adicionar a interface como serviço
builder.Services.AddScoped<ILivroRepository, LivroRepository>();
builder.Services.AddScoped<IEmprestimoRepository, EmprestimoRepository>();
builder.Services.AddScoped<IItemRepository, ItemRepository>();

builder.Services.Configure<CookiePolicyOptions>(options =>
{
    options.CheckConsentNeeded = context => true;
    options.MinimumSameSitePolicy = SameSiteMode.None;
});
```

HttpContextAccessor. Você pode usá-lo para obter detalhes sobre solicitações recebidas, incluindo cabeçalhos, cookies, parâmetros de consulta e declarações de usuários. Ao obter acesso à propriedade "Resposta", você também pode alterar a resposta HTTP

Adicionando as classes como

```
// Corrigir problema com TEMPDATA para aumentar o tempo de duração
builder.Services.AddDistributedMemoryCache();
builder.Services.AddSession(options =>
{
    // Set a short timeout for easy testing.
    options.IdleTimeout = TimeSpan.FromSeconds(900);
    options.Cookie.HttpOnly = true;
    // Deixar informado para o navegador que a sessão é essencial
    options.Cookie.IsEssential = true;
});
builder.Services.AddMvc().AddSessionStateTempDataProvider();

builder.Services.AddMemoryCache(); // Guardar os dados na memoria

//Add Gerenciador Arquivo como serviços
builder.Services.AddScoped<GerenciadorArquivo>();
builder.Services.AddScoped<EmprestimoLiivroAspCore.Cookie.Cookie>();
builder.Services.AddScoped<EmprestimoLiivroAspCore.CarrinhoCompra.CookieCarrinhoCompra>();

var app = builder.Build();
```